



# non-gon: A Non-Polygonal Collision Detection Library for Real-Time Queries Between Smooth Convex Shapes

**SOFTWARE  
METAPAPERS**

**IVO ROUPA**

**RODRIGO DAMASCENO**

**AASAM KARIM**

**LOURENÇO MOURÃO**

**JOÃO AFONSO**

**PEDRO CAMPOS**

**DANIEL SIMÕES LOPES**

*\*Author affiliations can be found in the back matter of this article*

**]u[**ubiquity press

## ABSTRACT

Accurate collision detection is fundamental to modern game engines and physics simulators, enabling applications in biomechanics, robotic obstacle avoidance, surgical training, and granular material simulation. While polygonal meshes remain the standard, smooth convex shapes, such as ellipses and ellipsoids, offer significant advantages in geometric accuracy, memory efficiency, and computational performance. Although analytical methods exist, they are scattered across disciplines and lack a unified implementation. To address this, we present **non-gon** a curated library comprised of 14 analytical functions for real-time distance and proximity queries between 2D and 3D smooth convex shapes, implemented in TypeScript and Three.js. In addition to the core analytic functions, the library features curve and surface rendering, along with several interactive web application examples. Finally, we invite all communities working on physics-based engines to contribute to the library, either by integrating its analytic functions into their codebases or by extending the collection of supported contact pairs.

## CORRESPONDING AUTHOR:

**Daniel Simões Lopes**

ITI / LARSyS, Instituto Superior Técnico, Universidade de Lisboa, Portugal

[daniel.s.lopes@tecnico.ulisboa.pt](mailto:daniel.s.lopes@tecnico.ulisboa.pt)

## KEYWORDS:

Smooth Convex Shapes; Collision Detection Library; Proximity Queries; Shortest Distance; Typescript; Three.js

## TO CITE THIS ARTICLE:

Roupa I, Damasceno R, Karim A, Mourão L, Afonso J, Campos P, Lopes DS 2026 **non-gon**: A Non-Polygonal Collision Detection Library for Real-Time Queries Between Smooth Convex Shapes. *Journal of Open Research Software*, 14: 61. DOI: <https://doi.org/10.5334/jors.585>

## (1) OVERVIEW

### INTRODUCTION

Collision detection determines whether objects are intersecting or separated in space, while also estimating the timing of contact and identifying the specific contact points on each object (1). It is widely applied in robotics (2–7), computational biomechanics (8, 9), autonomous systems (10, 11), granular simulations (12), video games (13–16), virtual reality (17–19), product prototyping (20), among other disciplines. By enabling precise contact force calculations, and contact point identification, collision detection algorithms accurately determine where and how contact forces should be applied, thereby ensuring realistic physical behaviors within physics-based simulations. There are two main types of collision detection methods: Proximity Query (PQ) and Shortest Distance (SD) (21). PQ algorithms determine whether two objects are in contact, or not, by analyzing their relative spatial configurations and geometric properties, focusing on detecting the collision state rather than computing exact distances (21). In contrast, SD methods compute the minimum distance between object boundaries, and the closest points of contact (22, 23). Thus, PQ methods are well-suited for the broad phase of collision detection, where fast, approximate checks are sufficient, while SD methods are better suited for the narrow phase, where precise measurements are essential—especially in physics-based simulations.

Due to their geometric versatility, mesh-based representations are the industry standard for modeling geometric collisions (24), yet they suffer from key limitations. Accurate collision detection involves computationally expensive distance checks between vertices, edges, and faces (23). In contrast, smooth convex representations offer a more geometrically accurate, memory efficient, and less resource-intensive alternative in scenarios where real-time interactions and simplified geometry is prioritized. Collision detection between smooth convex objects typically relies either on numerical or analytical approaches (25). Numerical approaches use optimization techniques or solve nonlinear systems of equations to provide accuracy-controlled results that are often difficult to solve analytically (24). However, achieving high degrees of precision with these methods often requires handling complex geometries or performing additional iterations, which can significantly increase computational cost and introduce convergence challenges (25). In contrast, analytical methods use exact, closed-form expressions derived from mathematical properties of geometric shapes to determine collision parameters. While they offer high accuracy and often faster computations, they are significantly less generalizable than numerical approaches (25). Analytical solutions must be individually

derived for each specific pair of shapes, making them difficult to formulate and limiting their availability in the literature (25). This complexity is a key reason why relatively few analytical collision detection methods exist.

Nevertheless, a number of geometric toolkits and physics-based engines incorporate analytic methods for resolving collisions between smooth, meshless, convex objects (26, 27). However, research on analytic collision detection for smooth convex shapes remains fragmented across multiple disciplines, and no comprehensive resource currently brings these techniques together in a unified toolkit. In this work, we introduce **non-gon** a curated library of analytic functions for real-time computation of shortest distances and proximity queries between smooth convex shapes. We review the mathematical foundations and practical implementations of 14 methods, covering both 2D and 3D geometries. **non-gon** offers accurate and efficient solutions for classic contact pairs, as well as support for proximity queries defined via algebraic conditions. To demonstrate its applicability, we provide interactive examples built with TypeScript (v5.7.2) and Three.js (v0.174.0), enabling real-time visualization of both proximity and shortest-distance computations in the browser.

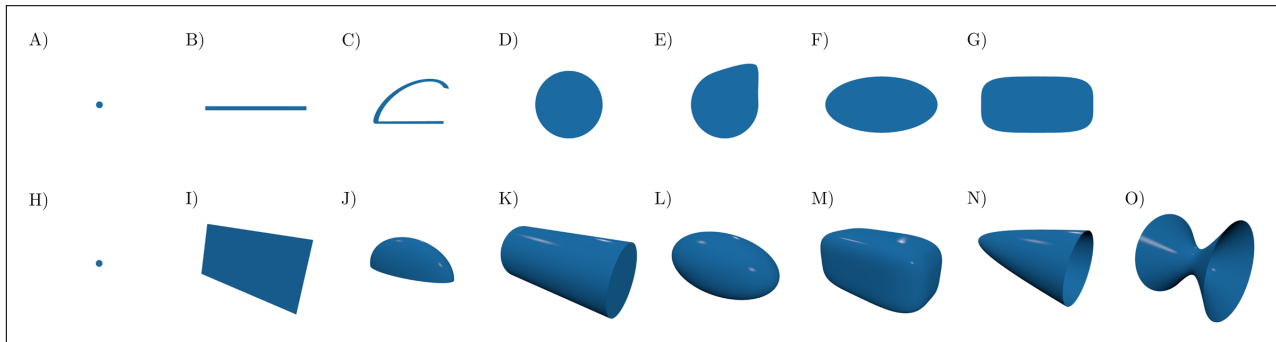
### SMOOTH CONVEX SHAPES AND CONTACT PAIRS

The **non-gon** library supports six 2D and eight 3D smooth convex shapes, as illustrated in Figure 1. Moreover, it includes implementations for five 2D SD queries, four 3D SD queries, one 2D PQ, and four 3D PQ contact pairs (Figure 2). For each contact pair, we analyzed the computational cost by calculating the number of floating-point operations (FLOPs) required by the corresponding PQ and SD algorithms (see Tables 1 and 2).

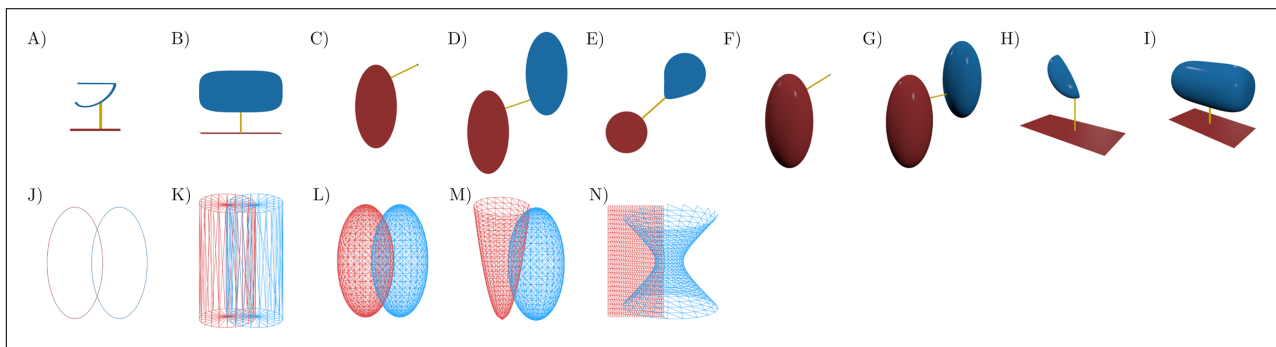
#### Shortest Distance

**Convex Line–Line Segment (Figure 2A):** We implemented the method proposed by Römer et al. (36) that uses normal parametrization to represent convex curves, mapping each boundary point to a unique outward normal. This approach ensures a well-defined representation for strictly convex shapes. The method describes the line segment implicitly and computes the closest points by aligning surface normals between the curve and the segment.

**Superellipse–Line Segment (Figure 2B):** Following the method reported by Lopes et al. (39), this algorithm exploits the geometric condition that the nearest point between a superellipse and a line segment occurs where their surface normals are parallel. The problem is reformulated to identify the point on the superellipse surface corresponding to a given outward normal. The superellipse is represented parametrically, while the line segment is described explicitly.



**Figure 1** All 2D (top) and 3D (bottom) shapes available in the **non-gon** library: **A)** 2D Point, **B)** Line Segment, **C)** Convex Line, **D)** Circle, **E)** Convex Circle, **F)** Ellipse, **G)** Superellipse, **H)** 3D Point, **I)** Plane, **J)** Cylinder, **K)** Ellipsoid, **L)** Superellipsoid, **M)** Elliptic Paraboloid, **N)** Hyperboloid.



**Figure 2** Shortest Distance (SD) and Proximity Query (PQ) 2D and 3D contact pairs available in the **non-gon** library: **A)** Convex Line-Line Segment (SD), **B)** Superellipse-Line Segment (SD), **C)** Point-Ellipse (SD), **D)** Ellipse-Ellipse (SD), **E)** Convex Circle-Circle (SD), **F)** Point-Ellipsoid (SD), **G)** Ellipsoid-Ellipsoid (SD), **H)** Hyperboloid-Plane (SD), **I)** Superellipsoid-Plane (SD), **J)** Ellipse-Ellipse (PQ), **K)** Cylinder-Cylinder (PQ), **L)** Ellipsoid-Ellipsoid (PQ), **M)** Ellipsoid-Elliptic Paraboloid (PQ), **N)** Hyperboloid-Plane (PQ).

| GEOMETRY PAIR                 | DIMENSION | MODEL                    | ADDITION | SUBTRACTION | MULTIPLICATION | DIVISION | ROOT | SINE/<br>COSINE | IF/ELSE |
|-------------------------------|-----------|--------------------------|----------|-------------|----------------|----------|------|-----------------|---------|
| Ellipse-Ellipse               | 2D        | Caravantes et al. (28)   | 32       | 36          | 162            | 2        | 0    | 4               | 3       |
| Ellipsoid-Ellipsoid           | 3D        | Caravantes et al. (28)   | 359      | 80          | 1313           | 3        | 0    | 12              | 1       |
| Cylinder-Cylinder             | 3D        | Chittawadigi et al. (29) | 82       | 293         | 324            | 66       | 32   | 0               | 188     |
| Ellipsoid-Elliptic Paraboloid | 3D        | Brozos et al. (30)       | 338      | 68          | 1280           | 3        | 0    | 12              | 1       |
| Hyperboloid-Plane             | 3D        | Bektas et al. (31)       | 3        | 8           | 7              | 1        | 0    | 0               | 5       |

**Table 1** FLOP count of the PQs implemented in the **non-gon** library, namely, ellipse - ellipse, ellipsoid - ellipsoid, cylinder - cylinder, ellipsoid - elliptic paraboloid, and hyperboloid - plane.

**Point-Ellipse (Figure 2C):** This method follows the approach of Schneider et al. (2), which reformulates the original constrained optimization problem into a univariate polynomial equation. The closest point is determined by solving this equation and selecting the root that minimizes the Euclidean distance. Both the point and the ellipse are treated as implicit representations in this formulation.

**Ellipse-Ellipse (Figure 2D):** Building on the point-to-ellipse distance technique proposed by Schneider et

al. (2), Ik-Sung Kim (33) solves the ellipse-ellipse SD case iteratively—starting from an initial point on one ellipse, the next closest point on the second ellipse is located. This point is then used as a new query to find a closest point back on the first ellipse. The process continues, alternating between the two shapes, until convergence on the closest pair is achieved.

**Convex Circle-Circle (Figure 2E):** Building on the same framework, we also implemented the method described by Römer et al. (36) that handles distance queries

| GEOMETRY PAIR                                      | DIMENSION | MODEL                | ADDITION  | MULTIPLICATION | DIVISION  | ROOT      | SINE/<br>COSINE |
|----------------------------------------------------|-----------|----------------------|-----------|----------------|-----------|-----------|-----------------|
| Point–Ellipse                                      | 2D        | Chou (32)            | 55        | 180            | 18        | 9         | 0               |
|                                                    | 2D        | Gordejuela et al.    | 94        | 420            | 8         | 4         | 0               |
|                                                    | 2D        | Kim (33)             | 68        | 103            | 11        | 7         | 0               |
|                                                    | 2D        | Chen et al. (34)     | 56        | 83             | 11        | 3         | 7               |
|                                                    | 2D        | Schneider et al. (2) | 28        | 52             | 8         | 5         | 0               |
| Ellipse–Ellipse                                    | 2D        | Kim (33)             | 81 + 112n | 65 + 75n       | 8 + 4n    | 10 + 9n   | 4 + 4n          |
| Ellipse–Ellipse (Distance of Closest Approach)     | 2D        | Zheng et al. (35)    | 82        | 153            | 38        | 20        | 0               |
| Superellipse–Straight line                         | 2D        | Lopes et al. (8)     | 22        | 32             | 12        | 6         | 8               |
| Convex Geometry–Straight line                      | 2D        | Römer et al. (36)    | 40        | 36             | 10        | 8         | 12              |
| Convex Geometry–Circle                             | 2D        | Römer et al. (36)    | 52        | 42             | 10        | 12        | 12              |
| Point–Ellipsoid                                    | 3D        | Schneider et al. (2) | 901       | 6              | 455       | 455       | 0               |
| Ellipsoid–Ellipsoid                                | 3D        | Kim (33)             | 85 + 910n | 65 + 15n       | 14 + 465n | 10 + 460n | 0               |
| Ellipsoid–Ellipsoid (Distance of Closest Approach) | 3D        | Zheng et al. (37)    | 70 + 142n | 123 + 190n     | 30 + 72n  | 16 + 28n  | 8n              |
| Superellipsoid–Plane                               | 3D        | Lopes et al. (8)     | 28        | 38             | 24        | 12        | 8               |
| Convex Geometry–Plane                              | 3D        | Römer et al. (38)    | 34        | 42             | 10        | 10        | 12              |

**Table 2** FLOP count of the SDs implemented in the **non-gon** library, namely point–ellipse, ellipse–ellipse, superellipse–straight line, convex geometry–straight line, convex geometry–circle, point–ellipsoid, ellipsoid–ellipsoid, ellipsoid–ellipsoid, superellipsoid–plane, and convex geometry.

between a circle and an almost-convex or concave shape using a potential-based generating function. The algorithm models the shape parametrically and the circle implicitly. It searches for the boundary point on the shape whose surface normal aligns with the circle’s varying normal direction, then calculates the distance between these two points.

**Point–Ellipsoid (Figure 2F):** Schneider et al. (2) extend their method for 3D, from point–ellipse to point–ellipsoid SD queries. This approach formulates the problem as a constrained optimization and solves it by reducing it to a single-variable polynomial, just as in the 2D case.

**Ellipsoid–Ellipsoid (Figure 2G):** Chen et al. (34) generalized the iterative closest-point method from ellipses to ellipsoids. Their algorithm alternates between the two surfaces by projecting onto each shape in turn until it converges to the closest pair of points, as we previously described for the *Ellipse – Ellipse* contact pair.

**Strictly Convex–Plane (Figure 2H):** Römer et al. (36) present a normal-parametrization technique extending the previously described for the *Smooth Convex – Line Segment* contact pair, where an extra degree of freedom was added to better capture both the surface normal and the boundary of the collision objects.

**Superellipsoid–Plane (Figure 2I):** Lopes et al. (39) present details on how to compute collision features

between a superellipsoid and a plane, which relies on surface-normal alignment to identify the shortest-distance contact points between a superellipsoid and a plane.

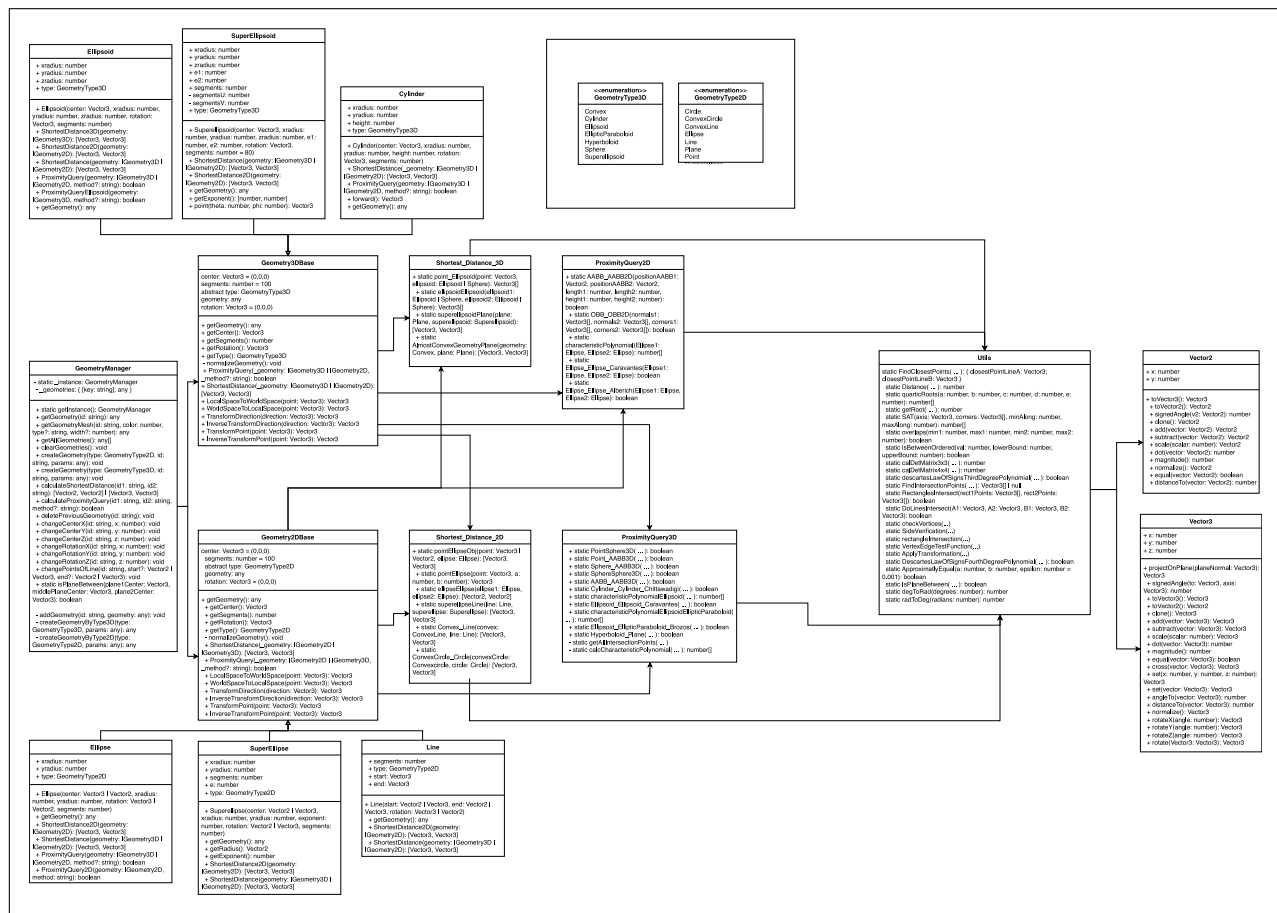
### Proximity Queries

**Ellipse–Ellipse (Figure 2J):** Caravantes et al. (28) propose a PQ algorithm that models intersection between ellipses (and between ellipsoids) as quadric equations to determine the collision state by analyzing the number of real roots of a characteristic polynomial derived from their combined system. This method enables a precise, algebraic classification of contact configurations between the ellipses (and ellipsoids).

**Cylinder–Cylinder (Figure 2K):** This PQ algorithm, based on Chittawadigi et al. (29), represents the cylinders using dual number algebra and the six Denavit–Hartenberg parameters (40) to compute the common normal and determine the collision state.

**Ellipsoid–Ellipsoid (Figure 2L):** This PQ algorithm extends the method by Caravantes et al. (28), previously described for the *Ellipse–Ellipse* contact pair, to handle 3D ellipsoidal shapes.

**Ellipsoid–Elliptic Paraboloid (Figure 2M):** The approach from Brozos et al. (30) models both shapes as quadrics and analyzes the characteristic polynomial’s coefficients to identify collision states.



**Figure 3** Architecture of the **non-gon** library showing class hierarchies, private, and public methods.

**Hyperboloid–Plane (Figure 2N):** Based on Bektas et al. (31), this method uses implicit quadric representations for both shapes and determines collision by solving their equations directly, avoiding iterative steps.

## IMPLEMENTATION AND ARCHITECTURE

The **non-gon** library, built with TypeScript (5.7.2) and Three.js (v0.174.0), organizes 2D and 3D geometries into distinct categories. Each shape type inherits from its own abstract base class to ensure consistency in structure and behavior across the entire system and less duplication of code.

A singleton base class, **GeometryManager** manages the instantiation of all shape types. Users can create geometries by providing a unique ID, an enum member to specify the desired geometry, and shape-specific parameters. All SD and PQ computations are encapsulated in dedicated public classes with static methods, allowing developers to invoke algorithms directly, without instantiating objects or using the singleton manager. Both algorithm types share a utility class that provides core mathematical routines required for execution.

This modular architecture—centered on abstract inheritance and a centralized singleton manager—enhances code reuse, simplifies maintenance, and

enables straightforward extensibility. **Figure 3** presents an overview of the complete architecture.

## Example Usage

General workflow for using the **non-gon** library involves instantiating two smooth convex shapes (in 2D or 3D) and computing their SD or PQ:

1. Initialize **Three.js** scene.
2. Initialize an object of class **GeometryManager()**.
3. Initialize shape 1 using **geometryManager.createGeometry()**.
4. Initialize shape 2 using **geometryManager.createGeometry()**.
5. Compute the shortest distance / proximity query between shapes using **geometryManager.calculateShortestDistance()** or **geometryManager.calculateProximityQuery()**.
6. Create shapes geometry using **geometryManager.createGeometry()**.
7. Add shapes geometry to scene.

The example below provides a minimal demonstration of how to create and render two ellipsoids. The line segment represents the shortest distance (SD) between their closest points (**Figure 4**).

```

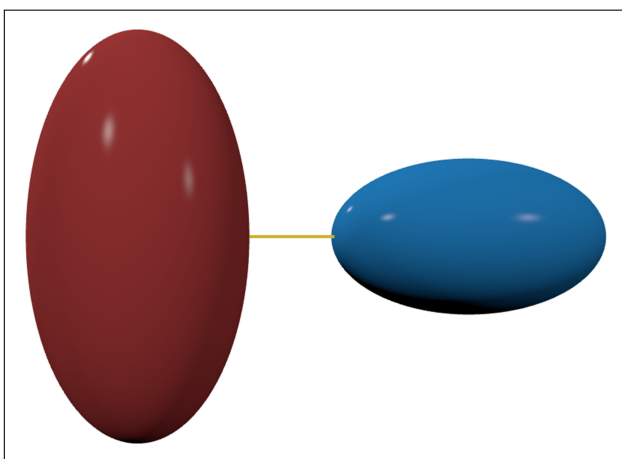
1  import * as THREE from "three";
2  import { GeometryManager, Vector3, GeometryType3D } from "non-gon";
3
4  // Initialize GeometryManager (if needed)
5  const geometryManager = GeometryManager.getInstance();
6
7  // Create renderer
8  const renderer = new THREE.WebGLRenderer({ antialias: true });
9  renderer.setSize(window.innerWidth, window.innerHeight);
10 document.body.appendChild(renderer.domElement);
11
12 // Set a visible background color
13 renderer.setClearColor(0xFFFFFF);
14
15 // Create scene
16 const scene = new THREE.Scene();
17
18 // Three-point light studio setup
19 // Key Light
20 const keyLight = new THREE.DirectionalLight(0xFFFFFF, 1.0);
21 keyLight.position.set(-100, 100, -100);
22 keyLight.target.position.set(0, 0, 0);
23 scene.add(keyLight);
24 scene.add(keyLight.target);
25
26 // Fill Light
27 const fillLight = new THREE.DirectionalLight(0xFFFFFF, 0.5);
28 fillLight.position.set(-100, 50, 100);
29 fillLight.target.position.set(0, 0, 0);
30 scene.add(fillLight);
31 scene.add(fillLight.target);
32
33 // Back Light
34 const backLight = new THREE.DirectionalLight(0xFFFFFF, 0.7);
35 backLight.position.set(100, 100, -75);
36 backLight.target.position.set(0, 0, 0);
37 scene.add(backLight);
38 scene.add(backLight.target);
39
40 // Create camera
41 const camera = new THREE.PerspectiveCamera(
42   75,
43   window.innerWidth / window.innerHeight,
44   0.1,
45   1000
46 );
47 camera.position.set(-125, 0, -50);
48 camera.lookAt(new THREE.Vector3(0, 0, 0));
49
50 let shape1 = 0x31AAFF; // blue
51 let shape2 = 0xE05050; // red
52 let connection = 0xC4A000; // yellow
53
54 let params0 = {
55   center: new Vector3(0, 0, 50),
56   xradius: 25,
57   yradius: 50,
58   zradius: 25,
59   rotation: new Vector3(90, 0, 0),
60   segments: 100
61 }
62

```

```

63 let params1 = {
64   center: new Vector3(0, 0, -50),
65   xradius: 25,
66   yradius: 50,
67   zradius: 25,
68   rotation: new Vector3(0, 0, 0),
69   segments: 100
70 }
71
72 geometryManager.createGeometry(GeometryType3D.Ellipsoid, "geo0", params0);
73 scene.add(geometryManager.getGeometryMesh("geo0", shape1, "mesh"));
74
75 geometryManager.createGeometry(GeometryType3D.Ellipsoid, "geo1", params1);
76 scene.add(geometryManager.getGeometryMesh("geo1", shape2, "mesh"));
77
78 let points = geometryManager.calculateShortestDistance("geo0", "geo1");
79 console.log("Shortest distance points:", points);
80 const lineMaterial = new THREE.LineBasicMaterial({ color: connection });
81 const lineGeometry = new THREE.BufferGeometry().setFromPoints([
82   new THREE.Vector3(
83     parseFloat(points[0].x.toFixed(3)),
84     parseFloat(points[0].y.toFixed(3)),
85     "z" in points[0] ? parseFloat(points[0].z.toFixed(3)) : 0
86   ),
87   new THREE.Vector3(
88     parseFloat(points[1].x.toFixed(3)),
89     parseFloat(points[1].y.toFixed(3)),
90     "z" in points[1] ? parseFloat(points[1].z.toFixed(3)) : 0
91   ),
92 ]);
93 const line = new THREE.Line(lineGeometry, lineMaterial);
94 scene.add(line);
95
96 // Animation loop
97 function animate() {
98   requestAnimationFrame(animate);
99   renderer.render(scene, camera);
100 }
101
102 animate();

```



**Figure 4** Surface rendering of two ellipsoids instantiated in the example code, with a line segment indicating the shortest distance between them.

## QUALITY CONTROL

The codebase includes a suite of unit tests implemented using the Jest testing framework. These tests are run via npm scripts, allowing for consistent test execution.

The testing strategy focuses on validating the SD and PQ implementations ensuring they are giving the expected result. To verify the correctness of the implementations, two complementary validation procedures were performed: (i) comparison against analytical benchmark cases with known solutions, and (ii) benchmarking against the GJK (for PQ) and combined GJK+EPA (for SD) algorithms as implemented in the COAL library (41).

For the analytical validation, four test cases were selected for each contact pair across both PQ and SD methods, covering two overlapping and two non-overlapping configurations. Each case was solved analytically, and the results were compared against the outputs produced by the NONGON implementation (see SUPPLEMENTARY MATERIAL section).

For the computational benchmark, NONGON's performance was evaluated using two datasets. The first dataset includes 2,000 cases per contact-pair category, evenly split between 1,000 overlapping and 1,000 non-overlapping configurations. The second dataset consists of 30 examples of edge and degenerate

configurations per contact pair type. Edge cases were defined as configurations in which the resulting  $\epsilon$  values, representing the computed distance between shapes, were either near-zero ( $\epsilon < 10^{-3}$ ) or excessively large relative to the characteristic dimensions of the geometries. For detailed information regarding the  $\epsilon$  values associated with each contact pair, please refer to [https://github.com/NON-GON/Testing-Dataset/blob/main/edge/EDGE\\_DATASET.md](https://github.com/NON-GON/Testing-Dataset/blob/main/edge/EDGE_DATASET.md).

Degenerate geometries were identified by evaluating the ratios between shape-defining parameters; configurations approaching limiting conditions ( $0.4 < r < 5$ ) that altered intrinsic dimensionality or compromised geometric validity were classified as degenerate. Parameters that do not affect intrinsic geometry, such as rotational degrees of freedom, were excluded from this criterion. Both datasets are available in .csv format in the **non-gon** Testing Dataset repository at <https://github.com/NON-GON/Testing-Dataset>.

The outputs produced by **non-gon** for the manually evaluated test cases, together with the corresponding results obtained using the GJK and GJK+EPA implementations from the COAL library (41), are provided in the Supplementary Material section. In addition, the complete results generated for the two generated datasets are available in the same repository.

### Limitations

The numerical implementation of the analytical formulations considered in this work provides solutions for a restricted class of smooth convex geometries while remaining computationally efficient. Nevertheless, numerical representations inherently introduce several challenges. These limitations arise from the underlying mathematical structure of the problem rather than from the current implementation of **non-gon**.

Specifically, numerical stability issues and floating-point exceptions, such as singularities from division by zero, undefined values (NaN), or overflow, can emerge in degenerate configurations or near-tangent contacts, where precision limits may compromise the reliability of the contact evaluation.

Furthermore, exact analytical solutions are typically available only for a limited set of contact pairs (e.g., point-ellipse, point-sphere, sphere-sphere, circle-circle, superellipsoid-plane, and certain ellipse-ellipse formulations based on characteristic polynomial approaches). For many practically relevant geometries, such as point-ellipsoid, ellipsoid-ellipsoid, cylinder-cylinder, superellipse-line, hemiellipsoid-plane, ellipsoid-elliptic paraboloid, and hyperboloid-plane pairs, proximity and shortest-distance computations rely on numerical techniques, particularly root-finding procedures.

Some of these issues are observable in the web-based demonstration across contact pairs, where overlapping

shapes do not always transition from a solid to a wireframe representation as expected, or where incorrect contact vectors are displayed between shapes.

## (2) AVAILABILITY

### OPERATING SYSTEM

**non-gon** runs on Unix-like, Windows and Mac operating systems.

### PROGRAMMING LANGUAGE

TypeScript ( $\geq 5.7.2$ )

### ADDITIONAL SYSTEM REQUIREMENTS

No special requirements.

### DEPENDENCIES

Three.js ( $\geq 0.174.0$ )

### LIST OF CONTRIBUTORS

Ivo Roupa, Rodrigo Damasceno, Asaam Karim, Lourenço Mourão, João Afonso, Pedro Campos, Daniel Simões Lopes

### SOFTWARE LOCATION

#### Archive

**Name:** non-gon

**Persistent identifier:** <https://www.npmjs.com/package/non-gon>

**Licence:** MIT License

**Publisher:** Rodrigo Damasceno

**Version published:** 0.0.1

**Date published:** 06/06/2025

#### Code repository: GitHub

**Name:** non-gon library

**Persistent identifier:** <https://github.com/NON-GON/NON-GON-Library>

**Licence:** MIT License

**Date published:** 06/06/2025

### LANGUAGE

English

## (3) REUSE POTENTIAL

**non-gon** is a library of analytic functions for computing shortest distances and proximity queries between 2D and 3D smooth convex shapes in real time. It provides efficient solutions for classic contact pairs such as point-ellipse, ellipse-ellipse, and ellipsoid-ellipsoid, and supports proximity queries based on algebraic conditions.

Beyond its standalone functionality, **non-gon** is designed to support integration into broader computational frameworks and interactive systems,

including physics engines, interactive visualization platforms, and computational mechanics applications such as the discrete element method and multibody dynamics.

To demonstrate these integration scenarios, **non-gon** was incorporated into two distinct physics engines: *matter-js* (<https://github.com/liabru/matter-js>) and *cannon-es* (<https://pmndrs.github.io/cannon-es/>), highlighting its adaptability across heterogeneous simulation environments.

### matter.js

The 2D *matter-js* example includes a multi-physics interactive system where analytic ellipses from *non-gon* are simulated using *matter-js* and visualized in a 2D canvas environment. In this setup, *matter-js* is responsible for rigid-body dynamics and boundary collisions, while analytic geometry operations from *non-gon* are used to evaluate precise ellipse–ellipse proximity relationships. Since exact smooth elliptical collision detection is not natively supported in *matter-js*, the system is extended with an external analytic collision layer that operates alongside the physics engine.

First, each simulated object is represented as a polygonal approximation within *matter-js* to enable standard rigid-body motion and collision response. In parallel, the same object is mapped to an analytic ellipse representation in *non-gon*, preserving its position, orientation, and geometric parameters. This mapping enables the computation of accurate geometric queries independent of the underlying polygonal physics representation. Second, a dedicated collision evaluation step is introduced that uses analytic proximity functions to detect intersections between ellipse pairs. These results are used to classify interacting objects and provide precise collision state information without altering the internal solver of *matter-js*.

### cannon-es

The 3D *cannon-es* example includes a multi-physics interactive system where analytic ellipsoids from *non-gon* are simulated in *cannon-es* and visualized in *Three.js*, with full bidirectional synchronization between geometry, physics, and rendering. Since such analytic ellipsoid shapes are not natively supported by *cannon-es*, the physics engine is extended to incorporate custom smooth convex shapes defined through analytic geometry. First, a custom ellipsoid shape is introduced by extending the engine’s native shape system. This shape wraps a corresponding analytic geometry representation and provides the required physical properties, including bounding volume estimation, inertia computation, and a simplified spatial bounding approximation. Second, the narrow-phase collision system is extended with a dedicated contact handler

for ellipsoid–ellipsoid interactions. This component leverages analytic geometry operations to evaluate potential intersections and compute closest-point information between shapes. These results are then used to construct physically consistent contact constraints, including contact normals and relative offsets required by the solver.

The presented examples demonstrate how **non-gon** can be seamlessly integrated into existing physics engines without modifying the core simulation engine, instead augmenting it with exact analytic computations for smooth convex shapes. For further details, please refer to the *cannon-es* integration example and the *matter-js* integration example, respectively: [cannon-es example](#) and [matter-js example](#). Additionally, we also provide a step-by-step “Quick Start” video, available at <https://non-gon.github.io/NON-GON-WEBSITE/video.html>, that covers the main prerequisites (e.g., Node.js), dependency installation, build and run instructions, and a minimal reproducible example.

## ADDITIONAL FILE

The additional file for this article can be found as follows:

- **Supplementary Materials.** Provides detailed results supporting the evaluation presented in the main manuscript. DOI: <https://doi.org/10.5334/jors.585.s1>

## ACKNOWLEDGEMENTS

The authors acknowledge the Portuguese Recovery and Resilience Program (PRR) for its financial support via IAPMEI/ANI/FCT under Agenda C645022399-00000057 (eGamesLab) and Fundação para a Ciência e a Tecnologia for its financial support through projects 10.54499/LA/P/0083/2020 and UID/50009/2025. We also acknowledge support from European Union’s COST Action CA22132, Open Network on DEM Simulations (ON-DEM), especially Work Group 1: Passing through time and space scales, for useful discussions and networking activities related to this work.

The authors are grateful for the initial code efforts developed by Raquel Agostinho, Inês Lúcio, Tiago das Neves and João Marto.

## FUNDING STATEMENT

Funding was provided by the Portuguese Recovery and Resilience Program (PRR) via IAPMEI/ANI/FCT under Agenda C645022399-00000057 (eGamesLab) and by Fundação para a Ciência e a Tecnologia through projects 10.54499/LA/P/0083/2020 and UID/50009/2025.

## COMPETING INTERESTS

The authors have no competing interests to declare.

## AUTHOR AFFILIATIONS

**Ivo Roupa**  [orcid.org/0000-0001-6344-9997](https://orcid.org/0000-0001-6344-9997)

ITI / LARSyS, Portugal

**Rodrigo Damasceno**  [orcid.org/0009-0002-3588-833X](https://orcid.org/0009-0002-3588-833X)

ITI / LARSyS, Instituto Superior Técnico, Universidade de Lisboa, Portugal

**Aasam Karim**  [orcid.org/0000-0002-7366-7127](https://orcid.org/0000-0002-7366-7127)

ITI / LARSyS, Portugal

**Lourenço Mourão**  [orcid.org/0009-0004-1424-7407](https://orcid.org/0009-0004-1424-7407)

ITI / LARSyS, Instituto Superior Técnico, Universidade de Lisboa, Portugal

**João Afonso**  [orcid.org/0009-0002-2301-6542](https://orcid.org/0009-0002-2301-6542)

ITI / LARSyS, Instituto Superior Técnico, Universidade de Lisboa, Portugal

**Pedro Campos**  [orcid.org/0000-0001-7706-5038](https://orcid.org/0000-0001-7706-5038)

Wow!Systems Informática Lda, Funchal, Portugal; ITI / LARSyS, Universidade da Madeira, Portugal

**Daniel Simões Lopes**  [orcid.org/0000-0003-0917-9396](https://orcid.org/0000-0003-0917-9396)

ITI / LARSyS, Instituto Superior Técnico, Universidade de Lisboa, Portugal

## REFERENCES

- Ericson C.** Chapter 2 - Collision Detection Design Issues. In: Ericson C, editor. *Real-Time Collision Detection. The Morgan Kaufmann Series in Interactive 3D Technology*. San Francisco: Morgan Kaufmann; 2005. pp. 7–21. Available from: <https://www.sciencedirect.com/science/article/pii/B9781558607323500073>.
- Schneider P, Eberly DH.** *Geometric tools for computer graphics*. Elsevier; 2002.
- Hadap S, Eberle D, Volino P, Lin MC, Redon S, Ericson C.** Collision Detection and Proximity Queries. In: *ACM SIGGRAPH 2004 Course Notes. SIGGRAPH '04*. New York, NY, USA: Association for Computing Machinery; 2004. p. 15–es. DOI: <https://doi.org/10.1145/1103900.1103915>
- Kockara S, Halic T, Iqbal K, Bayrak C, Rowe R.** Collision detection: A survey. In: *2007 IEEE International Conference on Systems, Man and Cybernetics*. IEEE; 2007. pp. 4046–4051. DOI: <https://doi.org/10.1109/ICSMC.2007.4414258>
- Moore M, Wilhelms J.** Collision detection and response for computer animation. In: *Proceedings of the 15th annual conference on Computer graphics and interactive techniques*. 1988. pp. 289–298. DOI: <https://doi.org/10.1145/54852.378528>
- Gregory A, Lin MC, Gottschalk S, Taylor R.** A framework for fast and accurate collision detection for haptic interaction. In: *ACM SIGGRAPH 2005 Courses*. 2005. pp. 34–es. DOI: <https://doi.org/10.1145/1198555.1198604>
- Jiménez P, Thomas F, Torras C.** 3D collision detection: a survey. *Computers & Graphics*. 2001;25(2):269–285. DOI: [https://doi.org/10.1016/S0097-8493\(00\)00130-8](https://doi.org/10.1016/S0097-8493(00)00130-8)
- Lopes D, Neptune R, Ambrósio J, Silva M.** A superellipsoid-plane model for simulating foot-ground contact during human gait. *Computer methods in biomechanics and biomedical engineering*. 2016;19(9):954–963. DOI: <https://doi.org/10.1080/10255842.2015.1081181>
- Arbabi E, Boulic R, Thalmann D.** Fast collision detection methods for joint surfaces. *Journal of Biomechanics*. 2009;42(2):91–99. DOI: <https://doi.org/10.1016/j.jbiomech.2008.10.017>
- Goudarzi P, Hassanzadeh B.** Collision risk in autonomous vehicles: classification, challenges, and open research areas. *Vehicles*. 2024;6(1):157–190. DOI: <https://doi.org/10.3390/vehicles6010007>
- Hamidaoui M, Talhaoui MZ, Li M, Midoun MA, Haouassi S, Mekkaoui DE, et al.** Survey of Autonomous Vehicles' Collision Avoidance Algorithms. *Sensors (Basel, Switzerland)*. 2025;25(2):395. DOI: <https://doi.org/10.3390/s25020395>
- Rubio-Largo S, Lind P, Maza D, Hidalgo R.** Granular gas of ellipsoids: analytical collision detection implemented on GPUs. *Computational Particle Mechanics*. 2015;2(2):127–138. DOI: <https://doi.org/10.1007/s40571-015-0042-y>
- Morgan G, Storey K.** Scalable collision detection for massively multiplayer online games. In: *19th International Conference on Advanced Information Networking and Applications (AINA'05) Volume 1 (AINA papers)*, vol. 1. Taipei, Taiwan; 2005. pp. 873–878. DOI: <https://doi.org/10.1109/AINA.2005.302>
- Ng K, Yeap Y, Tan Y, Ghauth K.** Collision detection optimization on mobile device for shoot'em up game. In: *2012 International Conference on Computer & Information Science (ICCIS)*, vol. 1. IEEE; 2012. pp. 464–468. DOI: <https://doi.org/10.1109/ICCISci.2012.6297290>
- Lazaridis L, Papatsimouli M, Kollias KF, Sarigiannidis P, Fragulis GF.** Hitboxes: A Survey About Collision Detection in Video Games. In: *International Conference on Human-Computer Interaction*. Springer; 2021. pp. 314–326. DOI: [https://doi.org/10.1007/978-3-030-77277-2\\_24](https://doi.org/10.1007/978-3-030-77277-2_24)
- Andrews S, Erleben K, Ferguson Z.** Contact and friction simulation for computer graphics. In: *ACM SIGGRAPH 2022 Courses*; 2022. pp. 1–172. DOI: <https://doi.org/10.1145/3532720.3535640>
- J. Li BG, Guo S.** Design of collision detection algorithms and force feedback for a virtual reality training intervention operation system. In: *2015 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, Zhuhai, China; 2015. pp. 1660–1665. DOI: <https://doi.org/10.1109/ROBIO.2015.7419010>
- Tzafestas C, Coiffet P.** Real-time collision detection using spherical octrees: virtual reality application. In: *Proceedings 5th IEEE International Workshop on Robot and Human Communication*. RO-MAN'96 TSUKUBA, Tsukuba, Japan;

1996. pp. 500–506. DOI: <https://doi.org/10.1109/ROMAN.1996.568888>
19. **Hubbard PM.** Interactive collision detection. In: *Proceedings of 1993 IEEE Research Properties in Virtual Reality Symposium*. IEEE; 1993. pp. 24–31. DOI: <https://doi.org/10.1109/VRAIS.1993.378267>
  20. **Held M, Klosowski JT, Mitchell JS.** Evaluation of collision detection methods for virtual reality fly-throughs. In: *Canadian Conference on Computational Geometry*. Citeseer; 1995. pp. 205–210.
  21. **Lin MC, Manocha D, Kim YJ.** Collision and proximity queries. In: *Handbook of discrete and computational geometry*. Chapman and Hall/CRC; 2017. pp. 1029–1056.
  22. **Johnson DE, Cohen E.** A framework for efficient minimum distance computations. In: *Proceedings. 1998 IEEE International Conference on Robotics and Automation (Cat. No. 98CH36146)*, vol. 4. IEEE; 1998. pp. 3678–3684. DOI: <https://doi.org/10.1109/ROBOT.1998.681403>
  23. **Gonçalves AA, Bernardino A, Jorge J, Lopes DS.** A benchmark study on accuracy-controlled distance calculation between superellipsoid and superovoid contact geometries. *Mechanism and Machine Theory*. 2017;115:77–96. DOI: <https://doi.org/10.1016/j.mechmachtheory.2017.04.008>
  24. **Huh S, Lee Y, Gornowicz G, Animation D, Henderson RD.** Fast and Robust Continuous Collision Detection (fastCCD). *Research Dreamworks Com*. 2014.
  25. **Beker O, Gürtler N, Shi J, Geist AR, Razmjoo A, Martius G, et al.** A Smooth Analytical Formulation of Collision Detection and Rigid Body Dynamics With Contact. *arXiv preprint arXiv:250311736*. 2025;. DOI: <https://doi.org/10.1109/IROS60139.2025.11247449>
  26. **Hofmann A, Mikelsons L, Gubsch I, Schubert C.** Simulating collisions within the Modelica multibody library. 2014. DOI: <https://doi.org/10.3384/ecp14096949>
  27. **Fiser D.** libccd: Library for collision detection between two convex shapes. URL: <https://github.com/danfis/libccd> (visited on 2023-07-05). 2018.
  28. **Caravantes J, Diaz–Toca GM, Fioravanti M, Gonzalez-Vega L.** Solving the interference problem for ellipses and ellipsoids: New formulae. *Journal of Computational and Applied Mathematics*. 2022;407:114072. DOI: <https://doi.org/10.1016/j.cam.2021.114072>
  29. **Chittawadigi RG, Saha SK.** An analytical method to detect collision between cylinders using dual number algebra. In: *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE; 2013. pp. 5353–5358. DOI: <https://doi.org/10.1109/IROS.2013.6697131>
  30. **Brozos-Vázquez M, Pereira-Sáez MJ, Souto-Salorio MJ, Tarrío-Tobar AD.** Classification of the relative positions between a small ellipsoid and an elliptic paraboloid. *Computer Aided Geometric Design*. 2019;72:34–48. DOI: <https://doi.org/10.1016/j.cagd.2019.05.002>
  31. **Bektas S.** On the Intersection of a Hyperboloid and a Plane. *International Journal of Discrete Mathematics*. 2017;2(2):38–42.
  32. **Chou CC.** A closed-form general solution for the distance of point-to-ellipse in two dimensions. *Journal of Interdisciplinary Mathematics*. 2019;22(3):337–351. DOI: <https://doi.org/10.1080/09720502.2019.1624064>
  33. **Kim IS.** An algorithm for finding the distance between two ellipses. *Communications of the Korean mathematical society*. 2006;21(3):559–567. DOI: <https://doi.org/10.4134/CKMS.2006.21.3.559>
  34. **Chen S, Xin S, He Y, Wang G.** The closest and farthest points to an affine ellipse or ellipsoid. *Tsinghua Science and Technology*. 2012;17(4):481–484. DOI: <https://doi.org/10.1109/TST.2012.6297594>
  35. **Zheng X, Palffy-Muhoray P.** Distance of closest approach of two arbitrary hard ellipses in two dimensions. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics*. 2007;75(6):061709. DOI: <https://doi.org/10.1103/PhysRevE.75.061709>
  36. **Romer UJ, Fidlin A, Seemann W.** Explicit analytical solutions for two-dimensional contact detection problems between almost arbitrary geometries and straight or circular counterparts. *Mechanism and Machine Theory*. 2018;128:205–224. DOI: <https://doi.org/10.1016/j.mechmachtheory.2018.05.018>
  37. **Zheng X, Iglesias W, Palffy-Muhoray P.** Distance of closest approach of two arbitrary hard ellipsoids. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics*. 2009;79(5):057702. DOI: <https://doi.org/10.1103/PhysRevE.79.057702>
  38. **Romer UJ, Fidlin A, Seemann W.** The normal parameterization and its application to collision detection. *Mechanism and Machine Theory*. 2020;151:103906. DOI: <https://doi.org/10.1016/j.mechmachtheory.2020.103906>
  39. **Lopes DS, Neptune R, Ambrósio J, Silva M.** A superellipsoid-plane model for simulating foot-ground contact during human gait. In: *Computer Methods in Biomechanics and Biomedical Engineering*, vol. 19, 2016. pp. 954–963. DOI: <https://doi.org/10.1080/10255842.2015.1081181>
  40. **Denavit J, Hartenberg RS.** A kinematic notation for lower-pair mechanisms based on matrices. 1955. DOI: <https://doi.org/10.1115/1.4011045>
  41. **Pan J, Chitta S, Manocha D, Lamiraux F, Mirabel J, Carpentier J, et al.** Coal: an extension of the flexible collision library. Feb. 2025.

---

#### TO CITE THIS ARTICLE:

Roupa I, Damasceno R, Karim A, Mourão L, Afonso J, Campos P, Lopes DS 2026 **non-gon**: A Non-Polygonal Collision Detection Library for Real-Time Queries Between Smooth Convex Shapes. *Journal of Open Research Software*, 14: 61. DOI: <https://doi.org/10.5334/jors.585>

**Submitted:** 06 June 2025    **Accepted:** 31 July 2026    **Published:** 04 September 2026

#### COPYRIGHT:

© 2026 The Author(s). This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International License (CC-BY 4.0), which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited. See <https://creativecommons.org/licenses/by/4.0/>.

*Journal of Open Research Software* is a peer-reviewed open access journal published by Ubiquity Press.